



Library Management System

Name :- Muskan Mewada
Reg .No. :-26MEI10065
Faculty Name :- E. Nirmala
Class Id : - 0739
Course Code :- 1021

INDEX

S. No.	Particulars	Page No.
1	Introduction & Abstract	3
2	Problem Statement, Objectives & Scope	4
3	Functional & Non-Functional Requirements	5
4	System Architecture & Design	6
5	Workflow & Use Case Design	8
6	Class, Sequence & Storage Design	9
7	Implementation Details & Subject Mapping	10
8	Testing, Validation & Results	12
9	Challenges, Learnings & Future Enhancements	13
10	Conclusion & References	14

Smart Library Management System

• A menu-driven console application developed in Python.
• Four functional areas: Book Management, Member Management, Issue & Return, and Reports.
• Object-oriented representation of Book and Member entities.
• Central Library class for business operations.
• JSON-based local storage for persistent records.
• Input validation and basic error handling.
• Unit tests for core add, issue and return workflows.

Project Features

Feature | Description

Book Management | Add, list and search books by title, author or category.
Member Management | Register members and display their issued-book count.
Issue Book | Issue an available book to a registered member and calculate a due date.
Return Book | Return an issued book and update member/book status.
Library Summary | Display total, available and issued books and total members.
Persistence | Store records in library_data.json.
Validation | Prevent empty values, invalid menu choices and invalid loan periods.

---

2. Problem Statement, Objectives & Scope

2.1 Problem Statement

Manual or poorly structured library records can make it difficult to know whether a book is available, who has borrowed it, or how many books are currently in the library. Searching for a particular book may also require checking multiple records. A small computerized system can centralize these operations and make the workflow easier to follow.

2.2 Objectives

• Create a structured Python application for common library operations.
• Apply data structures to store books and members.
• Use functions and modules to divide the program into manageable components.
• Apply OOP to represent books, members and library operations.
• Maintain book availability and issue information consistently.
• Provide validation and basic error handling.
• Demonstrate testing of important project operations.

2.3 Scope

In Scope | Out of Scope in Current Version

Book registration and search | Online/cloud synchronization
Member registration | Payment gateway
Issue and return | Multi-branch library management
Availability tracking | Advanced authentication/roles
Due date calculation | External database server
Local JSON persistence | Web/mobile interface
Summary reporting | Automated email/SMS notifications

2.4 Target Users

The primary target users for the academic prototype are small library staff or administrators. The system can also be demonstrated by students as an example of applying Python programming concepts to a real-world problem.

3. Functional & Non-Functional Requirements

3.1 Functional Requirements

ID | Requirement | Expected Behaviour

FR-01 | Add Book | Accept book ID, title, author and category and store a new book.
FR-02 | List Books | Display all stored books with their current status.
FR-03 | Search Books | Find books using title, author or category keywords.
FR-04 | Add Member | Register a member with ID, name and phone.
FR-05 | List Members | Display registered members and their issued-book counts.
FR-06 | Issue Book | Issue only an available book to an existing member.
FR-07 | Return Book | Return an issued book and restore availability.
FR-08 | Summary | Display total, available, issued books and total members.

3.2 Non-Functional Requirements

Quality | Requirement

Usability | Menu-driven interaction with clear prompts and messages.
Reliability | Invalid input should not crash normal application flow.
Maintainability | Separate files for models, business logic, storage, validation and reports.
Resource Efficiency | Uses lightweight local JSON storage and standard Python libraries.
Testability | Core library operations are covered by unit tests.
Error Handling | Missing records, duplicate IDs and invalid values are handled with messages.

3.3 Inputs and Outputs

Inputs: book details, member details, search keywords, menu choices and loan period. Processing: validation, searching, status changes, member-book association and due-date calculation. Outputs: confirmation messages, lists of books/members, issue/return results and library summary.

4. System Architecture & Design

4.1 Architecture Overview

The project follows a simple modular architecture. The user interacts with main.py. The Library class performs the main business operations. Book and Member classes hold entity data. The storage module reads and writes JSON data, while validators handles input checks and reports formats the summary output.

Architecture Diagram

Layer | Component | Responsibility

Presentation | main.py | Menu, prompts, user interaction and routing.
Business Logic | library.py | Book/member search, issue, return and summary operations.
Domain Model | models.py | Book and Member classes and serialization helpers.
Validation | validators.py | Non-empty and positive-integer input validation.
Reporting | reports.py | Human-readable library summary.
Persistence | storage.py | Load/save JSON records.
Data | library_data.json | Persistent books and members.

Textual Architecture Representation

User
↓
main.py — Menu / User Interface
↓
library.py — Core Business Logic
■ ↓ ■
models.py validators.py reports.py
↓
storage.py
↓
library_data.json

4.2 Design Principles

• Separation of responsibilities: each module has a focused purpose.
• Encapsulation: entity data and related serialization logic are grouped in classes.
• Validation before processing: user input is checked before library operations.
• Persistence after changes: add, issue and return operations save updated data.

---

5. Workflow & Use Case Design

5.1 Main Workflow

Step | Workflow

1 | Start application.
2 | Display main menu.
3 | User selects Book, Member, Issue/Return or Summary operation.
4 | System validates required input.
5 | Library class performs the requested operation.
6 | Updated records are saved to JSON where required.
7 | System displays result and returns to menu.
8 | User selects Exit to end the session.

Workflow Diagram (Text Form)

START
↓
Load JSON records
↓
Display Menu
↓
Select Operation
↓
Validate Input
↓
Perform Operation
↓
Save Updated Data
↓
Display Result
↓
Return to Menu / Exit

5.2 Use Cases

Actor | Use Case | Outcome

Library Staff | Add Book | New book becomes available in records.
Library Staff | Search Book | Matching books are displayed.
Library Staff | Add Member | Member becomes registered.

5.3 Workflow & Use Case Design

Actor | Use Case | Outcome

Library Staff | Issue Book | Book becomes Issued and member receives the book ID.
Library Staff | Return Book | Book becomes Available and member record is updated.
Library Staff | View Summary | Current library counts are displayed.

---

6. Class, Sequence & Storage Design

6.1 Class Design

Class | Attributes | Important Methods

Book | book_id, title, author, category, status, issued_to, due_date | to_dict(), from_dict()

Member | member_id, name, phone, issued_books | to_dict(), from_dict()
Library | books, members | add_book(), add_member(), search_books(), issue_book(), return_book(), summary()

6.2 Sequence: Issue Book

Sequence | Action

1 | User selects Issue Book in main.py.
2 | main.py receives Book ID, Member ID and loan period.
3 | Library.find_book() locates the book.
4 | Library.find_member() locates the member.
5 | System checks that the book is Available.
6 | Due date is calculated using the standard date library.
7 | Book status and member issued_books are updated.
8 | storage.save_data() writes the new state to JSON.
9 | Confirmation message is displayed.

6.3 Storage / ER-Style Design

Entity | Key Fields | Relationship

Book | book_id (PK), title, author, category, status, issued_to, due_date | A book may be linked to one member while issued.
Member | member_id (PK), name, phone, issued_books[] | A member may have zero or more issued book IDs.

The current version uses JSON rather than a relational database. Therefore, an ER diagram is represented conceptually through the Book–Member relationship in the stored JSON structure.

7. Implementation Details & Subject Mapping

7.1 Project File Structure

File | Purpose

main.py | Main menu and user interaction.
models.py | Book and Member OOP classes.

library.py | Core library business logic.
storage.py | JSON persistence.
validators.py | Input validation helpers.
reports.py | Library summary output.
tests/test_library.py | Unit tests.
README.md | Installation, usage and project overview.
statement.md | Problem statement, scope, target users and features.

7.2 VITyarthi Topic Mapping

Course Topic | Application in Project

Python Fundamentals | Variables, strings, lists, dictionaries and program structure.
Operators | Comparisons, arithmetic calculations and logical conditions.
Input & Output | Menu prompts, user input and formatted output.
Type Conversion | Conversion of loan period to integer.
Core Data Structures | Lists and dictionaries store books, members and issued-book IDs.
Control Flow | while loop, if/else and dictionary-based menu routing.
Functions | Separate functions for each user operation and validation.
Modules & Packages | Project is divided across multiple .py modules.
Array Data Structure | Python lists are used for collections of books and members.
OOP | Book, Member and Library classes encapsulate project entities/operations.
Look into the Future | Future enhancements are documented for GUI, database and authentication.

7.3 Key Algorithm: Issue Book

1. Find book by ID → 2) find member by ID → 3) reject missing records → 4) reject if book is already issued → 5) calculate due date → 6) set book status to Issued → 7)

add book ID to member list → 8) save JSON → 9) return confirmation.

---

8. Testing, Validation & Results

8.1 Testing Approach

Testing is included to verify that important operations work as expected. The project uses Python's built-in unittest framework so no external testing package is required.

Test ID | Scenario | Expected Result

TC-01 | Add a new book | Book count increases and book is stored.
TC-02 | Add a new member | Member count increases and member is stored.
TC-03 | Issue available book to valid member | Book status becomes Issued and due date is assigned.
TC-04 | Issue already issued book | Operation is rejected with an explanatory message.
TC-05 | Return issued book | Book status becomes Available and member list is updated.
TC-06 | Return available book | Operation is rejected.
TC-07 | Search by keyword | Matching title/author/category records are displayed.
TC-08 | Empty required input | Validator asks the user to enter a value.
TC-09 | Invalid loan period | Validator rejects zero/negative values.
TC-10 | Reload application | Previously saved JSON records are loaded.

8.2 Test Execution

Run the following command from the project directory:

python -m unittest discover -s tests -v

The included tests verify adding records and the complete issue/return lifecycle. For final submission, screenshots should be captured from the running application and inserted in this section.

8.3 Suggested Result Screenshots

• Main menu screen.
• Successful Add Book screen.
• Successful Add Member screen.
• Search result screen.
• Book Issue confirmation with due date.
• Book Return confirmation.
• Library Summary screen.
• Successful unit-test output.

---

9. Challenges, Learnings & Future Enhancements

9.1 Challenges Faced

• Maintaining consistent status between a Book object and the member's issued_books list.
• Handling invalid or empty user input without terminating the application.
• Separating business logic from user interface code.
• Persisting Python objects as JSON-compatible dictionaries.
• Designing a project that demonstrates multiple course concepts without unnecessary external dependencies.

9.2 Learnings & Key Takeaways

• Python functions make repeated operations easier to organize and test.
• Lists and dictionaries are useful for structured in-memory collections.
• OOP helps model real-world entities such as books and members.
• Modules make a project easier to read, maintain and extend.
• Validation improves reliability and user experience.

• Persistent storage allows application data to survive between runs.

9.3 Future Enhancements

Enhancement | Description

Graphical Interface | Add a Tkinter interface for easier navigation.
SQLite Database | Replace JSON with relational storage for larger datasets.
Authentication | Add librarian/admin login and role-based access.
Fine Calculation | Automatically calculate late-return fines.
Reservation | Allow members to reserve unavailable books.
Export Reports | Generate CSV/PDF reports for library activity.
Advanced Search | Filter by multiple fields and availability.
Cloud/Web Version | Expose library services through a web application in a later version.

The future enhancements are intentionally kept separate from the current implementation so the submitted project remains focused on the Python concepts covered in the course.

10. Conclusion & References

10.1 Conclusion

The Smart Library Management System demonstrates how fundamental Python concepts can be combined to solve a practical problem. The project contains multiple functional modules, a clear input-processing-output workflow, modular source files, object-oriented classes, persistent data storage, validation and unit tests.

The design also provides a clear foundation for future expansion. A GUI, database, authentication and additional reporting features can be added without changing the basic separation between user interaction, business logic, models and storage.

10.2 Requirement Coverage

College Requirement | Project Coverage

At least 3 functional modules | Book Management; Member Management; Issue & Return; Reports.
Clear input/output workflow | Menu → validation → library operation → result.
At least 4 non-functional requirements | Usability, reliability, maintainability, resource efficiency, testability and error handling.
5–10 meaningful modules/classes/files | Multiple Python modules plus Book, Member and Library classes.
Documentation | README, statement file and this report.
Testing | Unit tests for add and issue/return operations.
Design artefacts | Architecture, workflow, use-case, class, sequence and storage descriptions.

10.3 References

• VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines (college-provided PDF).
• Python standard library modules used by the project: json, datetime, pathlib and unittest.
• Project source files and README included with the submission package.

Declaration

I confirm that this project is prepared for academic submission and is intended to demonstrate the Python concepts covered in the associated VITyarthi course.